

GUÍA OPENCLAW

openclaw.json

El cerebro del sistema

Cómo leerlo · N1 Funciona y ya · N2 Lo personalizo · N3 Lo potencio · N4 En producción

N1

Funciona y ya

*auth, agents, plugins,
gateway*

N2

Lo personalizo

*channels, session,
messages, tools*

N3

Lo potencio

skills (Whisper) y browser

N4

En producción

*seguridad, costos,
escalabilidad*

Módulo 1 · KeroClow · www.openagent.lat

1

MÓDULO 1

openclaw.json*El cerebro del sistema*

I — ¿Qué es openclaw.json?

openclaw.json es el archivo de configuración principal de OpenClaw. Si OpenClaw fuera un auto, este archivo sería el tablero de control: desde acá definís qué modelo de IA usás, cómo se conecta con WhatsApp, qué permisos tiene, cuántas conversaciones puede manejar a la vez, y si puede usar el navegador o transcribir audios.

A diferencia de AGENTS.md o SOUL.md — que definen el comportamiento del agente — openclaw.json define la infraestructura. Es la primera cosa que lee OpenClaw cuando arranca.

 Regla de oro

Cada vez que cambias algo en este archivo, necesitás reiniciar OpenClaw para que los cambios surtan efecto. El agente no lo lee en caliente.

II — Cómo leerlo y editarlo sin romper nada

El archivo está en formato JSON. Eso significa que tiene una estructura muy estricta de llaves { }, dos puntos : y comas ,. Un solo carácter mal puesto impide que OpenClaw arranque.

Dónde está el archivo

```
# Ruta típica en el VPS
$ cat ~/.openclaw/openclaw.json

# Para editarlo
$ nano ~/.openclaw/openclaw.json
```

Antes de editar: siempre hacé una copia

```
cp ~/.openclaw/openclaw.json ~/.openclaw/openclaw.json.bak
```

Si algo sale mal, restaurás con: `cp ~/.openclaw/openclaw.json.bak ~/.openclaw/openclaw.json`

Cómo validar que no rompiste nada

Después de editar, antes de reiniciar OpenClaw, validá la sintaxis JSON con este comando:

```
$ node -e
"JSON.parse(require('fs').readFileSync('/root/.openclaw/openclaw.json', 'utf8'))";
console.log('OK')"
```

Si imprime OK, el archivo está bien formado. Si tira un error, te dice exactamente en qué línea está el problema.

III — El archivo completo de un vistazo

Este es tu openclaw.json real, con cada sección etiquetada:

```
"auth": { ... } // 🗝️ Quién puede acceder y con qué proveedor
"agents": { ... } // 🧠 Modelo de IA y comportamiento del agente
"tools": { ... } // 🔧 Herramientas habilitadas (web, fetch)
"messages": { ... } // 💬 Configuración de reacciones en grupos
"commands": { ... } // 📋 Comandos nativos del sistema
"session": { ... } // 📁 Alcance de la sesión de conversación
"channels": { ... } // 📱 Política de quién puede escribir al bot
"gateway": { ... } // 🖱️ Conexión local con el servidor
"skills": { ... } // 🎯 Integraciones externas (Whisper)
"plugins": { ... } // 🔗 Plugins activos (WhatsApp)
"browser": { ... } // 🌐 Navegador automatizado (Chrome)
```

NIVEL 1 — Funciona y ya

N1

Funciona y ya

auth · agents · plugins · gateway — lo mínimo para arrancar

Estas cuatro secciones son el corazón del sistema. Sin ellas, OpenClaw no arranca. Con solo estas bien configuradas, ya tenés un bot funcionando.

auth — Quién puede acceder

Define con qué servicio de IA se conecta OpenClaw y cómo se autentica. Tu configuración actual usa OpenRouter como intermediario.

Línea / Clave	Qué hace / Qué significa
<code>"auth": {</code>	Abre la sección de autenticación.
<code> "profiles": {</code>	Acá podés tener uno o más perfiles. Útil si tenés distintos proyectos con distintos modelos.
<code> "openrouter:default": {</code>	Nombre del perfil. Lo elegís vos — es solo una etiqueta.
<code> "provider": "openrouter"</code>	Le dice a OpenClaw qué servicio usar para llegar a los modelos de IA.
<code> "mode": "api_key"</code>	Método de autenticación: mediante API key. La key va en el archivo .env, no aquí.
<code> }</code>	Cierra el perfil.
<code> }</code>	Cierra la sección de profiles.
<code>}</code>	Cierra auth.

🔑 ¿Qué es OpenRouter y por qué usarlo?

En lugar de crear cuentas en OpenAI, Anthropic y Google por separado, OpenRouter te da acceso a todos desde un solo lugar con una sola API key. Pagás solo lo que usás, y podés cambiar de modelo en un segundo sin tocar infraestructura.

agents — El modelo y su comportamiento

Esta sección define qué modelo de IA usás, dónde guarda archivos temporales, y cuántas conversaciones puede manejar a la vez.

Línea / Clave	Qué hace / Qué significa
<code>"primary": "openrouter/openai/gpt-4o"</code>	El modelo principal. Formato: proveedor/empresa/modelo. Este es el primero que se intenta usar.
<code>"fallbacks": [...]</code>	Modelos de respaldo si el principal falla. Actualmente apunta al mismo — en producción conviene poner uno

```
"workspace":
"/root/.openclaw/workspace"
```

```
"compaction": "safeguard"
```

```
"maxConcurrent": 4
```

```
"subagents": { "maxConcurrent":
8 }
```

más económico.

Carpeta donde el agente guarda archivos temporales mientras trabaja — su "escritorio" interno.

Cuando la conversación es muy larga, el sistema la comprime. "safeguard" guarda un respaldo antes de comprimir.

Máximo de conversaciones simultáneas que el agente principal puede manejar a la vez.

Los subagentes (tareas paralelas creadas por el agente) pueden correr 8 procesos en simultáneo.



Idea: fallback económico para alto volumen

Si tu bot recibe muchas consultas, podés usar gpt-4o-mini como fallback. Es 10x más barato y suficiente para respuestas simples. Solo se activa si gpt-4o falla o está saturado.

plugins — Canales activos

Define qué canal de mensajería está habilitado. Tu configuración usa WhatsApp.

Línea / Clave

```
"plugins": {
  "entries": {
    "whatsapp": {
      "enabled": true
    }
  }
}
```

Qué hace / Qué significa

Abre la sección de plugins.

Lista de plugins disponibles.

El plugin de WhatsApp.

Lo activa. Cambiarlo a false desactiva el canal sin borrarlo.

Cierra whatsapp.

Cierra entries.

Cierra plugins.

gateway — Conexión con el servidor

El gateway es el puente técnico entre OpenClaw y WhatsApp. Acá definís el modo de conexión y las credenciales de acceso.

Línea / Clave

```
"mode": "local"

"auth": { "token": "... " }
```

Qué hace / Qué significa

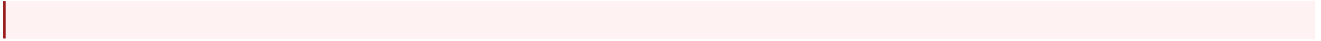
Conexión directa al servidor local. En producción casi siempre es "local".

El token de autenticación del servidor. Trátalo como contraseña — no lo compartas ni lo subas a GitHub.



El token del gateway es una contraseña

Ese string largo de letras y números es la llave de acceso al servidor. Si alguien lo obtiene, puede enviar mensajes desde tu bot. No lo publiques, no lo compartas, y considerá rotarlo periódicamente.



NIVEL 2 — Lo personalizo

N2

Lo personalizo

channels · session · messages · commands · tools

Con el Nivel 1 el bot funciona. El Nivel 2 define cómo se comporta: quién le puede escribir, cómo maneja las conversaciones, y qué tiene permitido hacer.

channels — Quién puede hablar con el bot

Esta es la sección más importante para controlar el acceso al bot. Desde acá definís si es abierto al público, solo para números específicos, o solo para grupos de tu lista.

Línea / Clave	Qué hace / Qué significa
"dmPolicy": "open"	Política para mensajes directos (1 a 1). "open" = cualquiera puede escribir. Otras opciones: "allowlist" para restringir.
"allowFrom": ["*"]	"*" significa cualquier número. Para restringir, reemplazá con: ["5491123456789", "5491198765432"]
"groupPolicy": "allowlist"	Para grupos: solo responde en grupos que vos autorices. Evita que el bot aparezca en grupos random.
"debounceMs": 0	Tiempo de espera entre que llega el último mensaje y el bot responde. 0 = responde inmediatamente.
"mediaMaxMb": 50	Tamaño máximo de archivos que el bot puede procesar. 50 MB es generoso para audios e imágenes.



Idea: debounce para clientes que escriben en partes

Muchos clientes escriben "hola" y después "quería consultar por el local de la calle X". Si dejás debounceMs en 0, el bot responde al primer mensaje solo. Con debounceMs: 1500 esperarás 1.5 segundos después del último mensaje antes de responder, capturando el mensaje completo.

session — Cómo se organiza la memoria de conversación

Define si el bot recuerda conversaciones por cuenta y canal por separado, o si mezcla contextos.

Línea / Clave	Qué hace / Qué significa
"dmScope": "per-account-channel-peer"	Cada combinación de (cuenta WhatsApp + canal + contacto) tiene su propia sesión independiente. Es el comportamiento correcto para bots con múltiples cuentas.

messages — Cómo reacciona en grupos

Controla en qué casos el bot reacciona con emojis a los mensajes dentro de grupos.

Línea / Clave	Qué hace / Qué significa
<code>"ackReactionScope": "group-mentions"</code>	Solo reacciona con emoji de confirmación cuando el bot es mencionado directamente en un grupo. Evita spam de reacciones en grupos activos.

commands — Comandos del sistema

Define qué comandos integrados de OpenClaw están disponibles y si se cargan automáticamente.

Línea / Clave	Qué hace / Qué significa
<code>"native": "auto"</code>	Carga automáticamente todos los comandos nativos del sistema (como /status, /reset, etc.).
<code>"nativeSkills": "auto"</code>	Carga automáticamente los comandos de los skills instalados.

tools — Herramientas habilitadas

Define si el agente puede hacer búsquedas web o traer contenido de URLs durante una conversación.

Línea / Clave	Qué hace / Qué significa
<code>"webSearch": { "enabled": false }</code>	La búsqueda web está deshabilitada. El bot no puede buscar en Google por su cuenta.
<code>"webFetch": { "enabled": false }</code>	El bot no puede acceder a URLs externas por su cuenta. Más seguro para bots de atención al cliente.

⚠ Cuándo habilitar webSearch

Si tu agente necesita buscar propiedades, precios o información en tiempo real, activá webSearch. Para un bot de atención inmobiliaria con su propia base de datos, lo mejor es mantenerlo deshabilitado y controlar qué información recibe vía tools personalizadas o SOUL.md.

NIVEL 3 — Lo potencio

N3

Lo potencio

skills (Whisper) · browser — capacidades adicionales

El Nivel 3 agrega capacidades que no son estrictamente necesarias para que el bot funcione, pero que mejoran significativamente lo que puede hacer.

skills — Integraciones externas

Los skills son integraciones con servicios externos. Tu configuración actual tiene habilitado Whisper, la API de transcripción de audio de OpenAI.

Línea / Clave	Qué hace / Qué significa
<code>"entries": {</code>	Lista de skills instalados.
<code> "openai-whisper-api": {</code>	El skill de transcripción de audios.
<code> "apiKey": "..."</code>	⚠ Esta API key debería estar en .env, no aquí. Ver Nivel 4.
<code> }</code>	Cierra el skill de Whisper.
<code>}</code>	Cierra entries.



Para qué sirve Whisper en un bot inmobiliario

Muchos clientes envían notas de voz en lugar de escribir. Sin Whisper, el bot los ignora o pide que escriban. Con Whisper activo, el bot transcribe el audio automáticamente y responde al contenido hablado — sin que el cliente tenga que hacer nada diferente.

browser — Navegador automatizado

OpenClaw puede controlar un navegador Chrome para hacer tareas web automatizadas: buscar propiedades, hacer scraping, completar formularios, tomar capturas de pantalla.

Línea / Clave	Qué hace / Qué significa
<code>"enabled": true</code>	El navegador está activo.
<code>"headless": false</code>	"false" = el navegador abre una ventana visual. En servidores sin pantalla, deberías usar "true".
<code>"noSandbox": true</code>	Necesario para correr Chrome en Linux sin privilegios de root. Casi siempre necesario en VPS.
<code>"executablePath": "/usr/bin/google-chrome"</code>	Ruta al ejecutable de Chrome en tu servidor.
<code>"cdpPort": 18800</code>	Puerto de control interno para que OpenClaw se comunique con Chrome. No necesitas tocarlo.



Idea: scraping automático de propiedades competencia

Con el browser habilitado, podés crear un tool que visite portales inmobiliarios, extraiga precios y disponibilidad, y los compare con tu cartera. El bot podría responder en tiempo real: 'El promedio de alquiler para locales en esa zona es \$X según los últimos 5 listados encontrados'.

NIVEL 4 — En producción

N4

En producción

Seguridad · optimización de costos · escalabilidad

Con los tres niveles anteriores tenés un bot completo. El Nivel 4 es lo que lo convierte en un sistema profesional: datos seguros, costos controlados, y listo para crecer.

Datos sensibles: qué va dónde

Tu openclaw.json actual tiene datos que deberían estar en el archivo .env, no en el JSON:

Dato	Está en	Debería estar en	Riesgo si queda en JSON
API key de OpenRouter	openclaw.json (vía .env)	archivo .env	Exposición si se sube a Git o se comparte el JSON
Token del gateway	openclaw.json	archivo .env	Acceso no autorizado al servidor
API key de Whisper	openclaw.json ⚠	archivo .env	Cargos inesperados si alguien la usa

🔑 **Cómo mover la API key de Whisper al .env**

1. Abrí el archivo .env: `nano ~/.openclaw/.env`
2. Agregá: `OPENAI_WHISPER_KEY=sk-tu-key-aquí`
3. En openclaw.json, reemplazá el valor de apiKey por: `"${OPENAI_WHISPER_KEY}"`
4. Reiniciá OpenClaw. El sistema reemplaza la variable automáticamente.

Optimización de costos con fallbacks

Actualmente tu fallback apunta al mismo modelo que el principal (gpt-4o). Un setup de producción más eficiente separa tareas simples de tareas complejas:

Configuración	Cuándo usar	Ahorro estimado
gpt-4o primary + gpt-4o-mini fallback	Bot de atención al cliente con preguntas variadas	~40% en conversaciones largas
gpt-4o-mini primary + gpt-4o fallback	Consultas simples con escalada para casos complejos	~70% en volumen normal
gpt-4o primary (sin fallback)	Configuración actual — calidad máxima, costo máximo	Referencia: 100%

Ajuste de concurrencia según tu VPS

El valor de maxConcurrent (actualmente 4) debería ajustarse según los recursos del servidor:

VPS	vCPUs	RAM	maxConcurrent recomendado	Subagents
Básico	1	2 GB	2	4
Estándar (tu config)	2	4 GB	4	8
Productivo	4	8 GB	8	16
Alto volumen	8+	16 GB+	16+	32+

VIII — Errores frecuentes y cómo resolverlos

OpenClaw no arranca después de editar el JSON

El archivo tiene un error de sintaxis. JSON no perdona: una coma de más, una llave sin cerrar, o unas comillas que faltan. Corrélo por el validador: `node -e "JSON.parse(require('fs').readFileSync('/root/.openclaw/openclaw.json','utf8'))"` — te va a decir exactamente en qué línea está el error.

El bot responde con error de autenticación

La API key de OpenRouter es incorrecta, expiró, o no tiene saldo. Verificá en openrouter.ai que la key siga activa y que haya crédito disponible. También puede ser que la variable de `.env` no esté cargando correctamente — reiniciá el servicio `systemd`.

El bot no procesa audios de WhatsApp

El skill de Whisper está configurado pero la API key es inválida, o el archivo de audio supera el límite de `mediaMaxMb`. Verificá la key en platform.openai.com y aumentá `mediaMaxMb` si recibís audios largos.

El bot responde en grupos donde no debería

`groupPolicy` está en `'open'` o hay un grupo que quedó en la `allowlist` por error. Revisá la sección `channels` y verificá la lista de grupos permitidos con el comando `/groups` desde el panel de admin.

El navegador (browser) falla o no abre

Chrome no está instalado en la ruta especificada, o `headless:false` está activado en un servidor sin interfaz gráfica. Verificá que Chrome esté en `/usr/bin/google-chrome` con: `which google-chrome`. En VPS sin pantalla, cambiá `headless` a `true`.

IX — Hoja de referencia rápida

Sección	Nivel	Función principal	Sin ella...
auth	N1	Proveedor de IA y autenticación	OpenClaw no puede conectarse a ningún modelo
agents	N1	Modelo, workspace y concurrencia	No sabe qué modelo usar ni dónde guardar archivos
plugins	N1	Canales activos (WhatsApp)	No hay canal de comunicación habilitado
gateway	N1	Conexión al servidor local	No puede comunicarse con el servidor
channels	N2	Control de acceso (quién escribe)	Cualquiera puede usar el bot sin restricciones
session	N2	Gestión de memoria por conversación	Las sesiones se mezclan entre contactos
messages	N2	Reacciones en grupos	Puede spamear con reacciones en grupos activos
commands	N2	Comandos del sistema disponibles	Los comandos nativos no cargan
tools	N2	Web search y fetch habilitados	Activos por defecto — sin restricciones web
skills	N3	Integraciones externas (Whisper)	No puede transcribir audios
browser	N3	Automatización de navegador Chrome	No puede navegar ni hacer scraping

→ Siguiente módulo: Workspace

openclaw.json define la infraestructura. El siguiente módulo es Workspace — cómo está organizada la carpeta de trabajo del agente, para qué sirve cada archivo, y cómo se estructuran los archivos que el agente lee en cada sesión.